

---

# DEEP IMPLICIT LAYERS FOR SMOOTH IMPLIED VOLATILITY SURFACE INTERPOLATION

---

 **Rafael Zimmer**

Institute of Mathematical Science and Computing  
University of São Paulo - Brazil  
rafael.zimmer@usp.br

 **Adalton Sena Filho**

Institute of Mathematical Science and Computing  
University of São Paulo - Brazil  
adaltonsena@usp.br

 **Luís Eduardo de Brito Câmara**

Institute of Mathematical Science and Computing  
University of São Paulo - Brazil  
eduardobritoc@usp.br

15th of July, 2024

## ABSTRACT

In the present paper, we implement a neural network architecture based on deep implicit layers with the objective of optimizing daily parameters for the SABR (stochastic alpha, beta, rho). The  $P, Q, R$  values are updated daily and used for generating smooth implied volatility surfaces in the context of financial options. The motivation behind our approach is to enable smooth interpolation and extrapolation of the surface, i.e., making it infinitely differentiable at all points while still maintaining some level of interpretability within our model. The main contribution of the proposed architecture is to solve the problem of selecting candidates  $\alpha, \rho, \nu$  which respective surface reduces arbitrage opportunities. Our methodology includes using historical observed values of  $\alpha, \rho, \nu$  and the innate ability of transformer architectures to process observations of sequences with non-fixed lengths, i.e., for contracts with varying expiration dates. The generated surfaces are then finally compared with results from classical interpolation methods such as splines and recurrent neural networks.

**Keywords** Neural Networks · Implied Volatility · Options · Quantitative Methods · Implicit Layers

# 1 Introduction

## 1.1 A Brief Introduction to Implied Volatility and Surface Reconstruction

Option contracts are one of the most important types of derivative contracts, and implied volatility surfaces play a critical role in the context of finding the correct no-arbitrage price [Ludkovski, 2023]. A derivative is an asset traded with respect to another asset (also called the underlying asset). An option specifically allows the buyer of the option to buy or sell (also called call and put contracts) a specified asset from the seller at a predefined future date. The pricing equations of these derivatives called the Black-Scholes model depend on the time to maturity of the contract, the price of the underlying asset to be traded, the price of the asset to be traded and the risk-free rate [Black and Scholes, 1973]. There is also an additional variable for contract pricing, called the implied volatility (*IVOL*). This value refers to the volatility considered, or to be expected, for the asset price in the future, and is an essential element in the Black-Scholes equation for pricing options contracts.

Having access to the true implied volatility values allows investors to make sales at the correct prices consistent with the actual no-arbitrage price of the contracts in the market. However, it is not always possible to obtain the *IVOL* values that fully reflect the characteristics of existing offers. Ensuring a way to obtain this value with confidence and readily available data in the market is a crucial task for quantitative analysts [Kim et al., 2006]. Traditional strategies for daily estimation of these values are predominantly based on interpolation algorithms [Homescu, 2011] or stochastic differential equation models [Jordan and Tier, 2011] that use numerical methods to approximate their values from existing option contracts in the market. Both methods try to approximate the correct value for the implied volatility for each pair of time to maturity and future asset price currently listed (also called strike price). This approach allows visualizing the model results in the form of a surface, where the **XY** axes are the maturity and strikes pairs and the **Z** axis is the *IVOL*. This surface is called the **implied volatility surface (IVS)**.

Ensuring accurate predictions of the IVS is crucial, as even small errors can lead to incorrect pricing and consequently large losses of profit or even negative return [Hagan et al., 2014] given the large leveraging capabilities of options. Traditional models such as linear interpolations, polynomial fitting, and *splines* generate arbitrage opportunities in the resulting IVS and often struggle to effectively capture market dynamics, besides lacking flexibility for extrapolation [Hagan et al., 2014].

## 1.2 Contributions of the Presented Work

In this paper, we propose an approach to address the challenges of interpolation and extrapolation of the IVS, reducing arbitrage opportunities. The chosen method uses a stochastic differential equation solution approach, specifically the *Stochastic Alpha, Beta, Rho (SABR)* model. Historical values of observed implied volatilities are used along with a nonlinear deep neural network to obtain parameters that minimize the surface error.

The optimization of the *SABR* model parameters is commonly performed using only a subset of the points observed daily [Kim et al., 2021], due to the fact that using all points introduces various outliers that do not match the real prices of the derivative, resulting in arbitrage opportunities in the final model. To solve this difficulty, parameters are often manually filtered according to criteria set by human experts [Hagan et al., 2002], and a subset of candidate points is used for parameter optimization. The use of models capable of receiving point clouds as input rather than manually allows the filtering step of candidate points to be much efficient and automated. The main contribution of this research will be the use of *neural implicit layers*, a variation of fully connected linear layers that solve a fixed point equation. This type of neural layers has been used efficiently for representing smooth densities and state-of-the-art performance when compared to *transformer* architectures [Kolter et al., 2020, Kawaguchi, 2021]. Unlike recurrent neural networks, the chosen architecture type is more suitable for handling unordered point clouds over time [Kim et al., 2024]. The optimization and filtering processes will be discussed in more detail under the development section.

## 1.3 The Transformer Architecture

The *SABR* model is a stochastic volatility model commonly used to obtain smooth surfaces, i.e., continuous and infinitely differentiable at all points [Hagan et al., 2015]. In the context of the *SABR* model, each candidate parameter will be assigned a score called *summarized suitability* score (*SS score*), which corresponds to the suitability of the candidates in terms of being summarized or represented by a subset of points, i.e., how feasible it is to reduce the number of candidates without altering the surface shape and increasing its smoothness. In summary, this score value is normalized and used to select only the points with the highest score, removing outliers that may introduce arbitrage in the optimization process of the *SABR* models.

Usually, the choice of subsets and the scoring process is done manually by analysts, as mentioned earlier, and the next section will focus on the details covering the *SABR* model and how we use a neural network with attention, i.e., a *Transformer* architecture, specifically the **encoder** layer for assigning scores automatically.

#### 1.4 Deep Implicit Layers

Differently from fully connected layers in which the transformation from input to output is defined explicitly, implicit layers are instead defined by conditions that the output must satisfy. For explicit layers, a given output  $z$  is computed directly from the input  $x$  using a function  $f$ , such that  $z = f(x)$ . In contrast, an implicit layer is the function  $g$  so that it depends on both the input  $x$  and the expected output  $z$ , where  $z$  is a solution for the equation  $g(x, z) = 0$ .

Formulating the problem as a fixed point equation can be used for a multitude of problems, including finding fixed points in recurrent networks, solutions to differential equations leading to Neural ODEs, and several optimization problems [Chen et al., 2019]. The implicit formulation offers several practical advantages, primarily the separation of the solution method from the layer definition which can be useful in maintaining interpretability of the resulting model [Kawaguchi, 2021].

Moreover, implicit layers offer significant benefits specifically in deep learning and automatic differentiation frameworks. Automatic differentiation methods store the entire computation graph, which can be memory-intensive. With implicit layers, however, the implicit function theorem allows computing gradients directly at the solution point and therefore storing intermediate variables becomes redundant [Massaroli et al., 2021]. This improves memory efficiency, making implicit layers particularly advantageous when substituting existing layers within large deep learning models [Li et al., 2020].

## 2 Implementation of a Parametric SABR Model and Implied Neural Network Architectures

### 2.1 Defining the SABR Model Equations

The SABR model is a stochastic differential model, defined by the following equations in which the dynamics of the forward price and the corresponding volatility levels are used to calculate the implied volatility.

- **Alpha** ( $\alpha$ ): This is the initial volatility level.
- **Beta** ( $\beta$ ): The elasticity parameter between the volatility of the asset and its price.
- **Rho** ( $\rho$ ): Statistical correlation between the asset price and its volatility.
- **Volvol** ( $\nu$ ): The volatility of the volatility process itself.

The implied volatility function of the SABR model is then given by:

$$\sigma_{BS}(f, K) = \alpha \frac{(fK)^{\frac{1-\beta}{2}}}{1 + \left( \frac{(1-\beta)^2 \rho^2}{24} + \frac{(1-\beta)^2 (2-3\rho^2)}{1920} \right) (fK)^{1-\beta}} \quad (1)$$

In the context of this paper, multiple SABR models are fitted daily, corresponding to each unique maturity value at the respective day. A continuous function is used to map the different maturities to the three model parameters (values for  $\beta$  are fixed and remain unchanged). The three variables used to encode the maturity levels are the following:

- **P**: Given a function  $f_\alpha(\cdot, \mathbf{P}) \rightarrow \mathbb{R}$ , the values for  $\alpha$  for each individual SABR model are encoded by  $\mathbf{P}$ , such that  $\mathbf{P} \in \mathbb{R}^5$ .
- **Q**: Given a function  $f_\rho(\cdot, \mathbf{Q}) \rightarrow \mathbb{R}$ , the values for  $\rho$  for each individual SABR model are encoded by  $\mathbf{Q}$ , such that  $\mathbf{Q} \in \mathbb{R}^4$ .
- **R**: Given a function  $f_\nu(\cdot, \mathbf{R}) \rightarrow \mathbb{R}$ , the values for  $\nu$  for each individual SABR model are encoded by  $\mathbf{R}$ , such that  $\mathbf{R} \in \mathbb{R}^4$ .

And finally, our daily parametric SABR model is given by the set of encoded parameters  $\{P, Q, R\}$  and the following equations to calculate values for  $f_\alpha$ ,  $f_\rho$ , e  $f_\nu$  at each unique maturity value  $t$ :

$$f_\alpha(t, p) = p_0 + \frac{p_3}{p_4} \left( \frac{1 - e^{-p_4 t}}{p_4 t} \right) + \frac{p_1}{p_2} e^{-p_2 t} \quad (2)$$

$$f_\rho(t, q) = q_0 + q_1 t + q_2 e^{-q_3 t} \quad (3)$$

$$f_\nu(t, r) = r_0 + r_1 t^{r_2} e^{r_3 t} \quad (4)$$

### 2.2 Arquitetura Transformer Utilizada e Etapas de Ajuste de Parâmetros

Transformers são um tipo de arquitetura de rede neural conhecida por sua capacidade de capturar dependências complexas em dados sequenciais. Neste projeto, um transformer encoder é usado para ajustar os parâmetros do modelo SABR ( $P$ ,  $Q$  e  $R$ ), correspondendo a  $\alpha$ ,  $\rho$  e  $\nu$  respectivamente. O transformer processa dados históricos de opções e produz estimativas de parâmetros que minimizam o erro entre as volatilidades implícitas do modelo e as volatilidades de mercado observadas.

Diferente das redes neurais recorrentes (RNNs) e das redes neurais convolucionais (CNNs), os transformers não dependem de uma estrutura sequencial para processar dados. Em vez disso, utilizam um mecanismo de atenção, que permite que cada elemento de entrada (ou token) considere a relação com todos os outros elementos na mesma sequência simultaneamente. Isso é realizado através de uma série de camadas chamadas de encoders (para codificação dos dados de entrada) e decoders (para gerar a saída).

Transformers são classificados como algoritmos de aprendizado de máquina devido à sua capacidade de aprender padrões e relações a partir de dados. Eles são treinados usando técnicas supervisionadas. No contexto da arquitetura específica que implementamos, utilizamos apenas as camadas do *encoder*, com 4 dimensões de entrada, 2 cabeças de atenção por camada e 4 camadas no total. A etapa de treino do modelo é realizada em cima de dados históricos de opções em cima do papel **S&P 500** para o ano de 2021-2022. As épocas de treino do modelo ajustam os pesos da

camada com uma função de erro que minimiza a diferença entre suas previsões e os valores das pontuações geradas manualmente. Essa capacidade de aprendizado é o que permite que transformers generalizem bem para novos dados não vistos durante o treinamento, fazendo previsões precisas baseadas nos padrões aprendidos.

A tarefa de ajuste de parâmetros do modelo SABR é especificamente uma tarefa de regressão. Em aprendizado de máquina, a regressão refere-se a problemas onde a saída é uma variável contínua, ao contrário da classificação, onde a saída é uma categoria discreta. No nosso caso, a saída do modelo transformer são as pontuações contínuas para cada parâmetro  $\alpha$ ,  $\rho$ , e  $\nu$  usados como entrada do algoritmo quasi-Newton de otimização não-linear Broyden-Fletcher-Goldfarb-Shanno (BFGS) [Broyden, 1970, Fletcher, 1970, Goldfarb, 1970, Shanno, 1970], tendo como alvo os valores  $\mathbf{P}$ ,  $\mathbf{Q}$  e  $\mathbf{R}$  que minimizem possíveis oportunidades de arbitragem de contratos com maturidades e *strikes* interpolados ou extrapolados.

## 2.3 Detalhes da Implementação

### 2.3.1 Classe do Modelo SABR

A classe ‘SABRModel’ implementa o modelo SABR, fornecendo métodos para calcular preços a termo, volatilidades implícitas e ajustar os parâmetros do modelo. Os métodos principais incluem:

- ‘star()’: Otimiza os valores  $P$ ,  $Q$  e  $R$  que minimizem o erro da superfície gerada com a observada dado valores fixos para tempos de vencimento, preço à vista e futuro, taxa livre de risco e rendimento de dividendos e  $\alpha$ ,  $\rho$ , e  $\nu$  variáveis.
- ‘ivol()’: Calcula a volatilidade implícita para parâmetros de modelo e condições de mercado dadas.

## 2.4 Arquitetura do Transformer

A arquitetura do transformer é conhecida por sua capacidade de capturar dependências complexas em dados sequenciais, utilizando um mecanismo de atenção para ponderar a importância de diferentes partes da sequência. A seguir, descrevemos a implementação de um modelo transformer em PyTorch, destacando os componentes principais: a atenção própria (Self-Attention), o bloco do transformer (Transformer Block) e o encoder do transformer (Transformer Encoder).

### 2.4.1 Atenção Própria

A classe `SelfAttention` implementa o mecanismo de atenção própria, que aceita dados contínuos e calcula a atenção para diferentes cabeças. A atenção é calculada através das matrizes de valor (`value`), chave (`key`) e consulta (`query`), resultando em uma saída ponderada pela importância de cada parte da sequência.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (5)$$

A implementação inclui:

```
class SelfAttention(nn.Module):
    def __init__(self, in_features, heads):

        self.head_dim = in_features // heads
        self.in_features = in_features
        self.heads = heads

        self.W_v = nn.Linear(self.head_dim, self.head_dim, bias=False)
        self.W_k = nn.Linear(self.head_dim, self.head_dim, bias=False)
        self.W_q = nn.Linear(self.head_dim, self.head_dim, bias=False)

        self.fc_out = nn.Linear(in_features, in_features)

    def forward(self, value, key, query, mask=None):
        N = query.shape[0]
        value_len, key_len, query_len = value.shape[1], key.shape[1], query.shape[1]
```

```

value = value.reshape(N, value_len, self.heads, self.head_dim)
key = key.reshape(N, key_len, self.heads, self.head_dim)
query = query.reshape(N, query_len, self.heads, self.head_dim)

values = self.W_v(value)
keys = self.W_k(key)
queries = self.W_q(query)

energy = torch.einsum("nqhd,nkhd->nhqk", [queries, keys])
if mask is not None:
    energy = energy.masked_fill(mask == 0, float("-1e20"))

attention = torch.softmax(energy / (self.head_dim ** (1 / 2)), dim=3)
out = torch.einsum("nhql,nlhd->nqhd", [attention, values]).reshape(N, query_len, self.in_features)

out = self.fc_out(out)
return out

```

### 2.4.2 Bloco do Transformer

A classe `TransformerBlock` implementa um bloco do transformer, composto por um mecanismo de atenção própria, normalização em camadas (`LayerNorm`) e uma rede feed-forward. Este bloco é a unidade básica que será repetida várias vezes no encoder:

```

class TransformerBlock(nn.Module):
    def __init__(self, in_features, heads, forward_expansion):
        self.attention = SelfAttention(in_features, heads)
        self.norm1 = nn.LayerNorm(in_features)
        self.norm2 = nn.LayerNorm(in_features)

        self.feed_forward = nn.Sequential(
            nn.Linear(in_features, forward_expansion * in_features),
            nn.ReLU(),
            nn.Linear(forward_expansion * in_features, in_features)
        )

    def forward(self, x, mask=None):
        attention = self.attention(x, x, x, mask)
        x = self.norm1(attention + x)
        forward = self.feed_forward(x)
        out = self.norm2(forward + x)
        return out

```

### 2.4.3 Encoder do Transformer

A classe `TransformerEncoder` implementa o encoder do transformer, composto por vários blocos do transformer. Este encoder processa a sequência de entrada, aplicando múltiplas camadas de atenção própria e redes feed-forward:

```

class TransformerEncoder(nn.Module):
    def __init__(self, in_features, heads, num_layers, forward_expansion, dropout, out_features):
        self.layers = nn.ModuleList([
            TransformerBlock(in_features, heads, forward_expansion) for _ in range(num_layers)
        ])

        self.dropout = nn.Dropout(dropout)
        self.fc_out = nn.Linear(in_features, out_features)

    def forward(self, x, mask=None):
        for layer in self.layers:
            x = layer(x, mask)

```

```
return self.fc_out(x)
```

### 3 Metodologia e Resultados

#### 3.1 Coleta de Dados

Os dados utilizados neste trabalho foram obtidos de dados históricos de opções sobre o ativo *SPY*, da bolsa estadunidense, de 2021 à 2022, disponível no Kaggle. Este conjunto de dados contém diversas colunas sobre opções negociadas, indexadas pela data, incluindo preços de exercício, preços futuros, maturidades, volatilidades implícitas, entre outros []. O arquivo pode ser e foi baixado e armazenado localmente em um diretório específico para processamento.

#### 3.2 Descrição do Conjunto de Dados

O conjunto de dados contém as seguintes colunas relevantes para nosso estudo, além de diversas outras:

- **underlying**: Preço do ativo subjacente.
- **strike**: Preço de exercício da opção.
- **daysToExpiration**: Dias restantes até a expiração da opção.
- **iv**: Volatilidade implícita.
- **dt**: Data do registro.

Além disso, adicionamos duas colunas calculadas:

- **maturity**: Calculada dividindo *daysToExpiration* por 360 (dias úteis no ano) para normalizar o tempo de maturidade.
- **r**: Taxa livre de risco obtida do Federal Reserve Economic Data (FRED) para o período correspondente.
- **d**: Taxa de dividendos paga trimestralmente pelo *SPY*.

#### 3.3 Pré-processamento dos Dados

O pré-processamento dos dados encorreu em várias etapas essenciais, predominantemente realizadas para permitir o uso dos dados diretamente na arquitetura do transformer e subsequente uso para gerar a superfície para os dados históricos. O transformer tem como entrada 4 dimensões, sendo elas:

1. O tempo até a maturidade;
2. O valor do parâmetro ( $\alpha$ ,  $\rho$ , ou  $\nu$ );
3. **1** se o parâmetro foi calculado para o dia atual, **0** se foi calculado no dia anterior por não existir no dia atual;
4. Parâmetros calculados utilizando as pontuações do dia anterior.

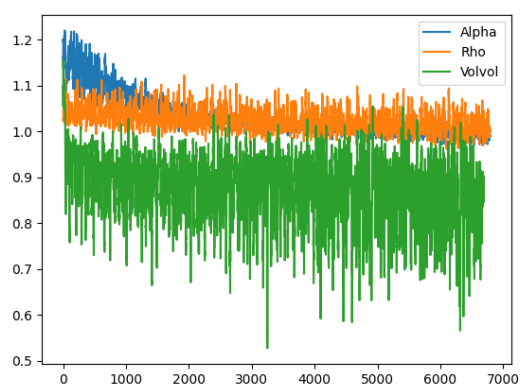
A variável target são os scores gerados utilizando-se todos os pontos candidatos para o dia atual, e a função de erro do modelo utiliza uma porcentagem **p** de pontos e é treinado para minimizar a função de erro quadrática das pontuações.

#### 3.4 Métricas de Erro e Comparação das Superfícies

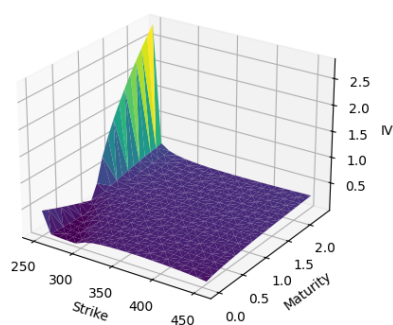
Utilizamos como baseline uma interpolação linear dos pontos observados diariamente, e comparamos com a performance do modelo em relação à pontos internos à uma malha (ou *grid* em inglês) gerada para pontos dentro do intervalo diário, tanto para valores de maturidade como de *strike*.

O nosso modelo transformer foi treinado em cima do conjunto de dados, de aproximadamente 70 observações, cada uma com uma sequência de em média 700 pontos. Os modelos foram treinados por aproximadamente 700 épocas.

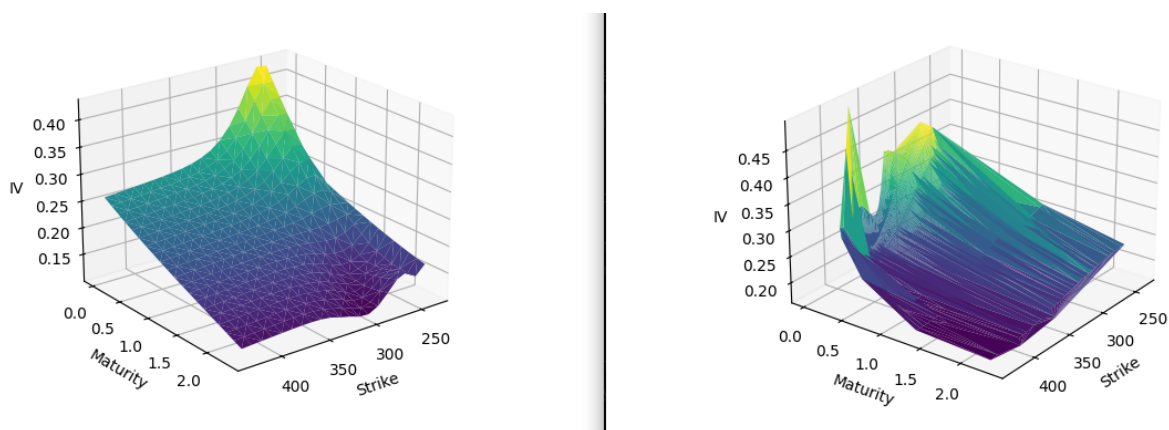




?figurename? 1: Função de perda histórica, de uma sessão de treinamento do Transformer



?figurename? 2: Superfície gerada utilizando um modelo SABR com parâmetros otimizados manualmente. Note que a não-remoção de outliers gerou um pico na superfície



?figurename? 3: Comparação da superfície real com a gerada após treinamento do Transformer. Na esquerda, a superfície do modelo SST, na direita, a superfície interpolada linearmente

## 4 Conclusão

### 4.1 Resumo

Este trabalho apresentou uma abordagem para ajustar os parâmetros do modelo SABR utilizando uma rede neural baseada na arquitetura de transformers (especificamente o elemento do *encoder*). A combinação do modelo SABR com a arquitetura transformer foi utilizada como forma de replicar os padrões da superfície de volatilidade implícita dia à dia, gerando a superfície suave e sem oportunidade de arbitragem para os contratos existentes observados no dia. O fato da escolha e pontuação dos candidatos ser possível de ser realizada com um modelo automático, em vez de manualmente, é uma solução extremamente eficiente, principalmente se considerarmos a possibilidade de utilizar esses modelos na GPU, o que permite a reconstrução da superfície em instantes, permitindo investidores identificar oportunidades de negociação em apenas poucos instantes, em vez de em uma escala diária.

### 4.2 Implicações e Trabalhos Futuros

O principal foco do trabalho é realmente a abordagem utilizando o transformer para substituir a escolha manual das pontuações para os candidatos, abrindo caminho para eventuais novas pesquisas e estratégias de High-Frequency Trading (ou *HFT*, abreviação em inglês). Pesquisas futuras podem explorar a integração de fatores de mercado adicionais e aprimorar a arquitetura que escolhemos, possivelmente aumentando a quantidade de blocos ou cabeças de atenção. Além disso, é importante considerar a escolha do otimizador dos parâmetros  $\mathbf{P}$ ,  $\mathbf{Q}$ ,  $\mathbf{R}$ , pois o tempo para realizar a etapa de pré-processamento foi um grande limitante no decorrer do trabalho. Estender o modelo para outros instrumentos financeiros também é uma possibilidade, pois permite o estudo sobre a adaptação do modelo a diferentes regimes de mercado e condições econômicas (tanto micro- como macro-econômicas) variáveis podem revelar novas oportunidades de aplicação e desenvolvimento do método proposto.

## ?refname?

- M. Ludkovski. Statistical machine learning for quantitative finance. *ANNUAL REVIEW OF STATISTICS AND ITS APPLICATION*, 10:271–295, 2023. ISSN 2326-8298. doi:10.1146/annurev-statistics-032921-042409.
- Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.
- Bo-Hyun Kim, Daewon Lee, and Jaewook Lee. Local volatility function approximation using reconstructed radial basis function networks. In J Wang, Z Yi, JM Zurada, BL Lu, and H Yin, editors, *ADVANCES IN NEURAL NETWORKS - ISSN 2006, PT 3, PROCEEDINGS*, volume 3973 of *Lecture Notes in Computer Science*, pages 524–530, HEIDELBERGER PLATZ 3, D-14197 BERLIN, GERMANY, 2006. Univ Elect Sci & Technol China; Chinese Univ Hong Kong; Asia Pacific Neural Network Assembly; European Neural Network Soc; IEEE Circuits & Syst Soc; IEEE Computat Intelligence Soc; Int Neural Network Soc; Natl Nat Sci Fdn China; KC Wong Educ Fdn, Hong Kong, SPRINGER-VERLAG BERLIN. ISBN 3-540-34482-9. 3rd International Symposium on Neural Networks (ISSN 2006), Chengdu, PEOPLES R CHINA, MAY 28-31, 2006.
- Cristian Homescu. Implied volatility surface: Construction methodologies and characteristics, 2011.
- Richard Jordan and Charles Tier. Asymptotic approximations to cev and sabr models. *SSRN Electronic Journal*, pages 1–38, May 2011. Available at SSRN: <https://ssrn.com/abstract=1850709> or <http://dx.doi.org/10.2139/ssrn.1850709>.
- Patrick S. Hagan, Deep Kumar, Andrew Lesniewski, and Diana Woodward. Arbitrage-free sabr. *Wilmott*, 2014(69): 60–75, 2014. doi:<https://doi.org/10.1002/wilm.10290>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/wilm.10290>.
- Hyeonuk Kim, Kyunghyun Park, Junkee Jeon, Changhoon Song, Jungwoo Bae, Yongsik Kim, and Myungjoo Kang. Candidate point selection using a self-attention mechanism for generating a smooth volatility surface under the sabr model. *EXPERT SYSTEMS WITH APPLICATIONS*, 173, JUL 1 2021. ISSN 0957-4174. doi:10.1016/j.eswa.2021.114640.
- Patrick Hagan, Deep Kumar, Andrew Lesniewski, and Diana Woodward. Managing smile risk. *Wilmott Magazine*, 1: 84–108, 01 2002.
- Zico Kolter, David Duvenaud, and Matt Johnson. Deep implicit layers - neural odes, deep equilibrium models, and beyond, 2020. NeurIPS 2020 tutorial. Retrieved from <https://implicit-layers-tutorial.org/>.
- Kenji Kawaguchi. On the theory of implicit deep learning: Global convergence with implicit layers. 2021. URL <https://arxiv.org/abs/2102.07346>.
- Sooan Kim, Seok-Bae Yun, Hyeong-Ohk Bae, Muhyun Lee, and Youngjoon Hong. Physics-informed convolutional transformer for predicting volatility surface. *QUANTITATIVE FINANCE*, 24(2):203–220, JAN 9 2024. ISSN 1469-7688. doi:10.1080/14697688.2023.2294799.
- Patrick Hagan, Andrew Lesniewski, and Diana Woodward. *Probability Distribution in the SABR Model of Stochastic Volatility*, volume 110, pages 1–35. 06 2015. ISBN 978-3-319-11604-4. doi:10.1007/978-3-319-11605-1\_1.
- Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. 2019. URL <https://arxiv.org/abs/1806.07366>.
- Stefano Massaroli, Michael Poli, Jinkyoo Park, Atsushi Yamashita, and Hajime Asama. Dissecting neural odes. 2021. URL <https://arxiv.org/abs/2002.08071>.
- Xuechen Li, Ting-Kam Leonard Wong, Ricky T. Q. Chen, and David Duvenaud. Scalable gradients for stochastic differential equations. 2020. URL <https://arxiv.org/abs/2001.01328>.
- Charles G Broyden. The convergence of a class of double-rank minimization algorithms 1. general considerations. *IMA Journal of Applied Mathematics*, 6(1):76–90, 1970.
- Roger Fletcher. A new approach to variable metric algorithms. *Computer Journal*, 13(3):317–322, 1970.
- Donald Goldfarb. A family of variable-metric methods derived by variational means. *Mathematics of Computation*, 24(109):23–26, 1970.
- David F Shanno. Conditioning of quasi-newton methods for function minimization. *Mathematics of Computation*, 24(111):647–656, 1970.